

差多少事先不知道：平时几行，出故障时几百万行；跨城带宽也贵。

能不能用最低的成本，把差异找出来修复？

我们的答案

找出两边哪几行不一样，网络传输量只跟差异多少有关，跟表多大无关。

出故障时，花费几 KB、用一个来回 RTT就能估出差了多少；最多两个来回找全差异。

解决什么问题：端到端一致性

没人能回答两端是否一致

链路跨机房、跨引擎、跨团队，每一段都说自己没问题，最终用数据的人无法确认。

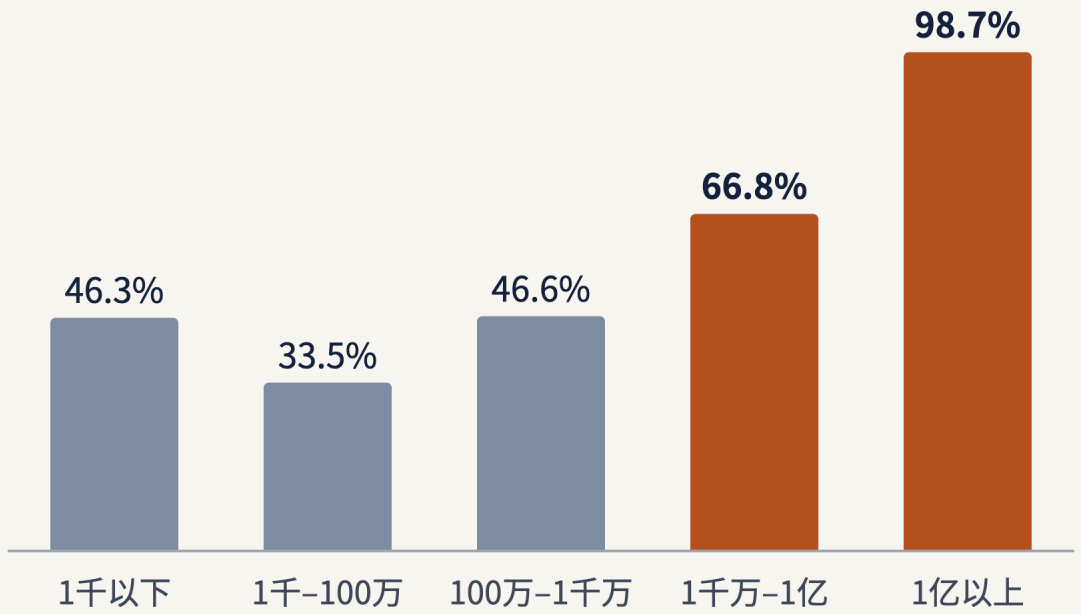
现有做法在大表上太贵

全量比对随表的规模增长；Merkle 一致时快，差异一多就要整块回传。偏偏大表几乎总有差异（右图）。

我们让它做得起

两端只交换指纹摘要，不搬原始数据；“一致性凭证”可以作为传输方的交付物。

生产中有差异的对账比例，按表的行数分组



橙色：1000 万行以上的表，只占对账次数的 6%，却占 77% 的总耗时

右图：NineData 对账服务 90 天、41,603 次表级对账（arXiv）

出故障时：一个来回，先知道差了多少

发现故障 → 差 50 条，还是几千万条？ → 决定怎么止血、要不要启动灾备评估

本方法

几KB，一个来回

还没开始修复，就能立刻估出差了多少行。

6 亿行的表实际差 93,130 行：16 KB 估计 8.4–10.1 万。

Merkle / 分区 checksum

比完才知道

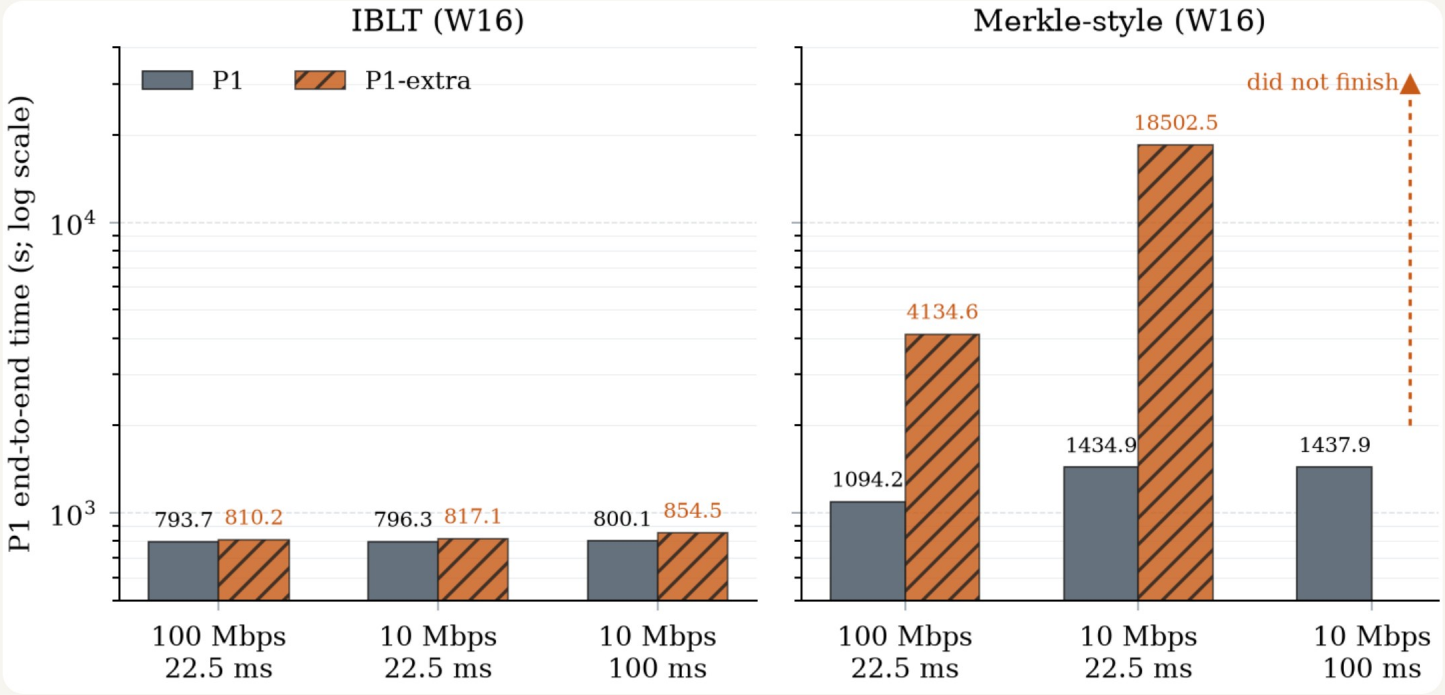
只知道哪些分区不一致；差多少行，要逐块下钻、全部比完才知道。

同一张表差异分散时，10 Mbps 下要 5 小时以上。

知道差多少之后，第二轮一次性定好参数，最多两个来回修复全量差异。

Oracle→MySQL （6.09 亿行）；生产 90 天中差异从 1 行到 4400 万行，跨 7 个数量级

特别擅长：大表、差异多、带宽紧、扫描贵



6.09 亿行的表，15.4 万处差异分散在整张表里（橙色斜纹）。

817 s
对 18,503 s

10 Mbps 下；延迟加到 100 ms，Merkle 跑不完。

只扫一遍、不要求按主键有序，分库分表和 hash 分片的库同样适用；Merkle 下钻要按范围回查。

本方法的耗时约等于两端各扫一遍库。

Oracle→MySQL （左组 P1：差异集中；右组 P1-extra：差异分散）

在哪里跑过

生产 · 运行 1 个月

NineData
Oracle→MySQL 对账 (v1)

零调参

差异从 0 到 13,310 行，自动适应
434 次全部成功，90% 一个来回结束

跨城生产链路 · 10 Mbps

中国移动
Redis / Pika (v1)

少 740 倍

传输量 141.7 KB vs Merkle 的 105.3 MB（差异 937 行）
4 万处突发差异时 80 s vs 523 s

存储系统原型

JuiceFS
4000 万个文件 (v2)

内存少90%

一次运行替代 gc + fsck，100 s vs 270–313 s
原有方法 Bloom Filter 每档漏约 1% vs 本方法 0 遗漏；

随机 IBLT(v2)：两端各压成指纹摘要，网络传输相减



摘要的大小只取决于**差了多少行**，与表有多大无关。

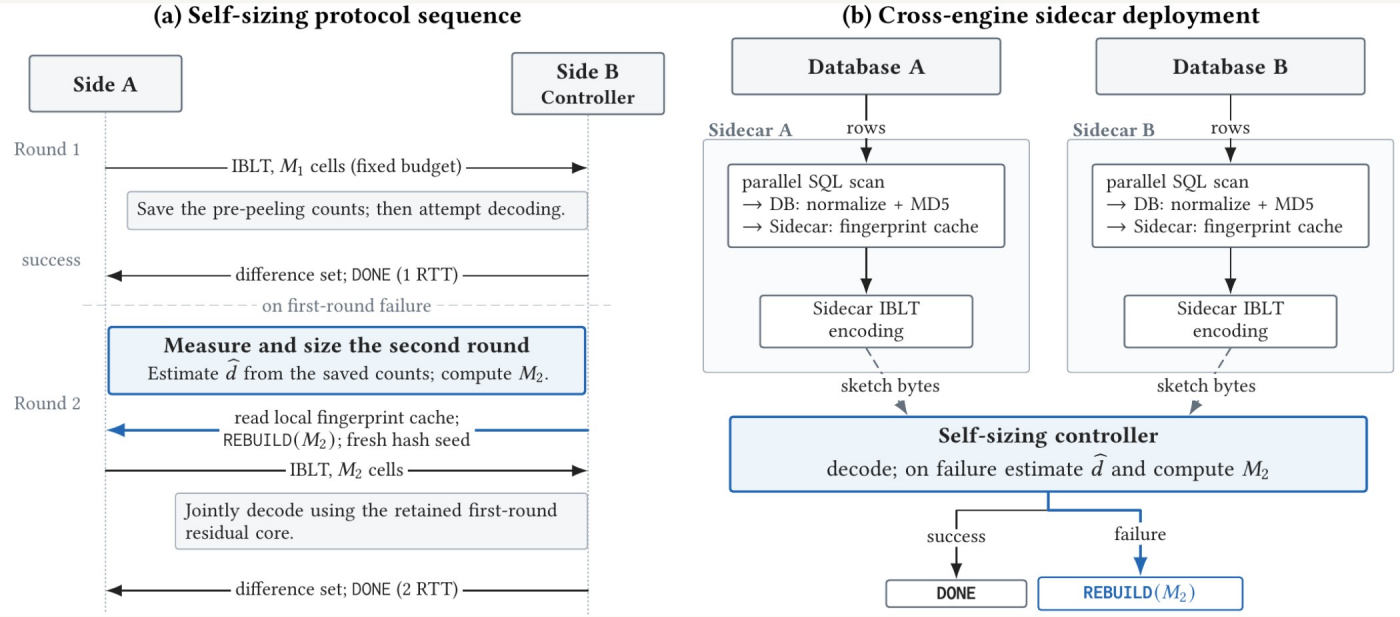
旁路部署、只读扫描；网络上只走摘要和最终差异。

与差异方向无关：同步里最常见的是单向差异（目标端漏数、源端多数）。旧方法要按方向离线标定，新方法免标定。

重复写入也能精确解出：分批、重试、多遍扫描都不怕。

摘要的结构和大小不变。

部署架构：两端各一个旁路进程



左：两轮协议，第一轮解开即结束。右：两端只读扫描、本地算指纹，跨机房只传摘要（sketch bytes），由控制器解出差异。

跨引擎的类型差异（NULL、数值、时间、字符集）在旁路进程里统一规范化后再算指纹，是工程实现，已在 Oracle→MySQL、Redis→Pika 上验证。

联合研究

真实场景做实验，合写顶会论文

产品集成

放进数据集成方案